

PyCS Tutorial

Table of Contents

1. Environment Setup	1
1.1. Tutorial	1
1.2. Common Services Network	1
1.3. Python	2
2. Connecting a client to the network	3
2.1. Basic connection	3
2.2. Waiting for registration	5
2.3. Registration timeout	6
2.4. Monitoring registration	6
2.5. Explicit start and stop	7
2.6. CommonServicesNode	7
3. Streetlights Example Definitions	9
3.1. csmf.msg.tutorial	9
3.1.1. Capability	10
Message urn:csmf:tutorial:capability version 1.0	10
3.1.2. Status	11
Message urn:csmf:tutorial:status version 1.0	11
3.1.3. StreetlightCommandRequest	12
Message urn:csmf:tutorial:streetlightcommandrequest version 1.0	12
3.1.4. StreetlightCommandResponse	13
Message urn:csmf:tutorial:streetlightcommandresponse version 1.0	13
4. Generating a message set	14
4.1. Generating Microformats	14
4.2. Generating Messages	14
5. Streetlights and Control applications	15
5.1. Streetlights Application	15
5.2. Control Application	16
6. Capability and Status	17
6.1. Publishing	17
6.2. Populating	18
6.3. Updating	19
6.4. Subscribing	21
7. Requests and Responses	23
7.1. Sending a Request	23
7.2. Handling Requests	23
Appendix A: Full Source Code	27
A.1. Connection	27
A.1.1. Streetlights	27

A.1.2. Streetlights Control	28
-----------------------------------	----

Chapter 1. Environment Setup

1.1. Tutorial

All source files for this tutorial are distributed within the `pys-tutorial.tgz` tar file. Untar this file into your working directory.

Extract the tutorial files

```
$ tar xzf pys-tutorial.tgz
$ ls pys-tutorial
README.md    definitions  docker      screenshots  src          tutorial.pdf
```

Alternatively, they may be found by cloning the `pys-tutorial` git repository, along with some extra files for generating this documentation.

Clone pys-tutorial git repository

```
$ git clone https://bitbucket.tdx.gss.gov.uk/scm/wfr/pys-tutorial.git
```

All files required for this tutorial will then be available in the `pys-tutorial` directory.

1.2. Common Services Network

All examples in this tutorial will assume a Common Services hub named `cs-hub` running on `localhost`. If you have access to the harbor docker image repository, a hub can be started by running the docker compose file inside this tutorial.

Start CS Hub docker container

```
$ ( cd pys-tutorial/docker && docker compose up --detach)
```

Give this a few minutes to start up and settle down.

Attach a web browser to <http://localhost:8051> to view the SILKWAVE console. Login using user 'admin' and password 'silkwave'. This console gives you an overview of the network at the low-level SILKWAVE transport level. We will not be using this console further during the tutorial, but it can be a useful tool in debugging SILKWAVE network issues.

Attach a web browser to <http://localhost:5154> to view the CS Monitor. This gives an overview of Common Services clients attached to the network. If all is running smoothly at this stage, you should see similar to the following:

[CS Monitor Screenshot] | [screenshots/cs-monitor.png](#)

Figure 1. CS Monitor Screenshot

If you see the four `silkwave`, `cs-registration`, `cs-capability` and `cs-status` processes running under

Process Status, and capability and status registrants in the Admin Interface table, then you have successfully started your hub.

1.3. Python

It is recommended that you use a python virtual environment to work with PyCS. For simplicity here we will use `venv`, but other managers like Coanda and Pipenv should work well depending on your preference or IDE support.

Set up python virtual environment

```
$ python -m venv venv
$ source venv/bin/activate
```

All commands from now on will assume that you have activated this virtual environment, as indicated by `(venv)$` in the command line prompts from now on. Your prompt may differ depending on how your shell is configured.

The tutorial depends the `pycs.client` package and its dependencies to run, and these must be installed into your virtual environment. If you do not have access to the pypi repository containing the relevant packages, you will need to either run one locally containing the relevant packages, (see <https://packaging.python.org/en/latest/guides/hosting-your-own-index/>), or manually install each package and its dependencies from the tar files.

Install pycs client package

```
(venv)$ pip install pycs.client
```

Chapter 2. Connecting a client to the network

Now we have an environment set up with a hub running, we can connect our first PyCS client.

All source files for this section can be found in the directory `pys-tutorial/src/connect_client`.

2.1. Basic connection

Create a `basic_client.py` file as follows.

basic_client.py

```
1 Unresolved directive in ../partials/connect.adoc - include::src/connect_client/basic_client.py[]
```

When the module is invoked, it simply calls the `main()` function, which is decorated with the `pys.client.client_application` decorator.

The first thing the `@client_application` decorator does is configure a number of command line parameters which will prove generally useful in setting up a common services application. We want to override the default values for some of these for our application, and so supply a `cs_id` for our CS network identifier, and `name` for our 'friendly' name on the network. We also hide these from the command line using `hidden`, so that users do not override them. As the hub that we are connecting to does not have any digital signatures enabled, we turn off this checking by default.

Show command usage

```
(venv)$ python basic_client.py --help
Usage: basic_client.py [OPTIONS]
```

Options:

<code>--sw-hub-name TEXT</code>	Name of the Silkwave hub to connect to. [silkwave.hub.name] [default: cshub]
<code>--sw-hub-host TEXT</code>	Hostname of the Silkwave hub to connect to. [silkwave.hub.host] [default: localhost]
<code>--sw-hub-port TEXT</code>	Port number of the Silkwave hub to connect to. [silkwave.hub.port] [default: 61623]
<code>--ssl-cert-file TEXT</code>	PEM file containing certificate, or certificate and key for SILKWAVE connection [silkwave.hub.cert_file]
<code>--ssl-key-file TEXT</code>	PEM file containing key for SILKWAVE connection [silkwave.hub.key_file]
<code>--ssl-key-password TEXT</code>	Password to decrypt ssl key [silkwave.hub.key_password]
<code>--ssl-ca-certs TEXT</code>	PEM file containing trusted certificates, or path to OpenSSL truststore directory [silkwave.hub.ca_certs]
<code>--username TEXT</code>	Username to authenticate with broker [silkwave.hub.username]
<code>--password TEXT</code>	Password to authenticate with broker [silkwave.hub.password]
<code>--cert FILE</code>	X509 Certificate for signing messages. [dsig.cert]
<code>--key FILE</code>	Key for signing messages. [dsig.key]
<code>--passphrase TEXT</code>	Passphrase of Key. [dsig.passphrase]
<code>--ca-pem FILE</code>	PEM File containing CA certificates. [dsig.ca_pem]
<code>--ca-dir DIRECTORY</code>	Directory containing CA certificates. [dsig.ca_dir]
<code>--reg-cert FILE</code>	X509 Certificate of the registration service. [dsig.reg_cert]
<code>--check / --no-check</code>	Check certificates on incoming message. [dsig.check]
<code>--allow-unsigned / --no-allow-unsigned</code>	Allow unsigned incoming messages. [dsig.allow_unsigned]
<code>--validate-incoming / --no-validate-incoming</code>	Validate incoming messages. [validation.incoming]
<code>--validate-outgoing / --no-validate-outgoing</code>	

```

--classification TEXT          Validate outgoing messages. [validation.outgoing]
--sci-controls TEXT           Default message security classification. [security.classification]
--dissemination-controls TEXT Default message security SCI controls. [security.sCIcontrols]
--rel-to TEXT                 Default message security dissemination controls. [security.disseminationControls]
--access-controls TEXT        Default message security releasable to. [security.releasableTo]
--owner-producer TEXT         Default message security access controls. [security.accessControls]
--creator-org TEXT            Default message security owner producer. [security.ownerProducer]
--classified-by TEXT           Default message security creator organization. [security.creatorOrganization]
--classification-reason TEXT   Default message security classified by. [security.classifiedBy]
--derived-from TEXT            Default message security classification reason. [security.classificationReason]
--declass-date TEXT            Default message security derived from. [security.derivedFrom]
--declass-exception TEXT       Default message security declass date. [security.declassDate]
--sw-classification TEXT       Default message security declass exception. [security.declassException]
--sw-sci-controls TEXT         Default SILKWAVE message security classification.
                               [silkwave.client.security.classification]
--sw-dissemination-controls TEXT Default SILKWAVE message security SCI controls. [silkwave.client.security.sCIcontrols]
--sw-rel-to TEXT               Default SILKWAVE message security dissemination controls.
                               [silkwave.client.security.disseminationControls]
--sw-access-controls TEXT      Default SILKWAVE message security releasable to.
                               [silkwave.client.security.releasableTo]
--sw-owner-producer TEXT       Default SILKWAVE message security access controls.
                               [silkwave.client.security.accessControls]
--sw-creator-org TEXT          Default SILKWAVE message security owner producer.
                               [silkwave.client.security.ownerProducer]
--sw-classified-by TEXT        Default SILKWAVE message security creator organization.
                               [silkwave.client.security.creatorOrganization]
--sw-classification-reason TEXT Default SILKWAVE message security classified by.
                               [silkwave.client.security.classifiedBy]
--sw-derived-from TEXT         Default SILKWAVE message security classification reason.
                               [silkwave.client.security.classificationReason]
--sw-declass-date TEXT         Default SILKWAVE message security derived from. [silkwave.client.security.derivedFrom]
--sw-declass-exception TEXT     Default SILKWAVE message security declass date. [silkwave.client.security.declassDate]
--sw-auth-id TEXT              Default SILKWAVE message security declass exception.
                               [silkwave.client.security.declassException]
--alt-label TEXT               Default SILKWAVE message security auth id. [silkwave.client.security.authID]
--auth-id TEXT                 Default SILKWAVE message security alt label. [silkwave.client.security.altLabel]
--alt-label TEXT               Default message security alt label. [security.altLabel]
--config FILENAME              Default message security auth id. [security.authID]
--log-config FILENAME          Default message security alt label. [security.altLabel]
                               YAML file containing default options and process configuration
--help                          YAML file containing logging configuration (see
                               https://docs.python.org/3/library/logging.config.html#logging-config-dictschema)
                               Show this message and exit.

```

The decorator then wraps our `main()` function in a call to `asuncio.run()`, adding signal handlers to shut the application down gracefully in the event of a SIGINT, SIGTERM or SIGHUP signal or an unhandled application exception. This effectively turns the `async main()` function into a synchronous call so it can be invoked directly in synchronous context.

Finally, the decorator uses the values from the command line arguments to initialise an instance of a `CommonServicesClient` and pass it to our function, passing the `kind` parameter though to the constructor. This client has already been started by the time we receive it, and will automatically disconnect and shut down when our function returns.

We are now ready to run our client application and connect to the Common Services network.

PyCS uses the python logging framework liberally. The default configuration will suffice for many purposes, but can be overridden by supplying the `--log-config` parameter and a YAML configuration file, as specified in <https://docs.python.org/3/library/logging.config.html#logging->

[config-dictschema](#). For this tutorial we will use the following configuration, which matches the default configuration apart from the log message format for clarity in the tutorial command output. You may of course leave this command line option off if you are happy with the default configuration, or adjust the settings to your requirements.

log-config.yml

```
Unresolved directive in ../partials/connect.adoc - include::src/log-config.yml[]
```

Run the application

```
(venv)$ python basic_client.py --log-config log-config.yml
INFO      : silkwave.async_client           : Connected to SILKWAVE as net:cshub:0058817451
INFO      : pycs.client.registration       : Registration accepted
INFO      : pycs.client.registration       : Deregistration successful
INFO      : silkwave.async_client           : Disconnected
```

From these logs you can see that our **Starting** log message is printed as the application enters our `run_client()` function.

In the background to this, the client is attempting to connect to the network. First of all it connects to the SILKWAVE hub, and then registers with the Common Services Registration Service.



As the connection is running asynchronously alongside our function, the order of log messages may be slightly different. Perhaps some stages of the connection will already have succeeded by the time our function is entered.

On the CS Monitor console Admin Interface section, make sure Service 'CS-Registration', Command 'list registrants' is selected, and click 'Go' You should see your client appear in the list of registrants below.

[CS Monitor Registrant List] | [screenshots/cs-monitor-client-connected.png](#)

After waiting 20 seconds, our **Stopping** log message is printed, and control returns out of our `main()` function. At this point the client deregisters from the Registration Service, and disconnects from the SILKWAVE hub. Refreshing the registrant list should see your client disappear.

Feel free to change the 20 second sleep to something longer if there is not enough time to see what you want to see.

2.2. Waiting for registration

You will have noted from the log message above, that at the point that we enter our `run_client()` function, the client may not have finished registering with the network. It is often useful to know programatically when this has happened so that we can continue with further processing. This can be done by awaiting `client.registered()`, which will return once the registration with the Common Services network has been successful.

Copy `basic_client.py` as `await_reg.py` and replace the `run_client()` function as follows:

await_reg.py

```
1 Unresolved directive in ../partials/connect.adoc - include::src/connect_client/await_reg.py[tag=show]
```

This time the log messages indicate that our application has been notified of the registration.

```
(venv)$ python await_reg.py --log-config log-config.yml
INFO      : silkwave.async_client      : Connected to SILKWAVE as net:cshub:1681896750
INFO      : pycs.client.registration   : Registration accepted
INFO      : pycs.client.registration   : Deregistration successful
INFO      : silkwave.async_client      : Disconnected
```

2.3. Registration timeout

We might also want to timeout if our client does not connect successfully within a certain time. This can be achieved using standard python `asyncio` mechanisms.

Copy `basic_client.py` as `timeout.py` and replace the `run_client` function as follows, and specify an invalid port number for the client creation.

timeout.py

```
1 Unresolved directive in ../partials/connect.adoc - include::src/connect_client/timeout.py[tag=show]
```

Run the application again, but supply an invalid port number for the SILKWAVE connection using the `--sw-hub-port` command line parameter so that the client is unable to connect.

```
(venv)$ python timeout.py --sw-hub-port 60000 --log-config log-config.yml
WARNING: stomp.py      : could not connect to host localhost, port 60000
ERROR    : silkwave.transport.stomp.fsm : Failed to connect transport
WARNING: stomp.py      : could not connect to host localhost, port 60000
ERROR    : silkwave.transport.stomp.fsm : Failed to connect transport
WARNING: stomp.py      : could not connect to host localhost, port 60000
ERROR    : silkwave.transport.stomp.fsm : Failed to connect transport
```

Here the client attempts to connect to the hub a number of times before our timeout expires, and we exit the application.

If you are using python 3.11 or later, then the `asyncio.timeout()` context manager may offer a more flexible option.

2.4. Monitoring registration

Of course, just because the client registered successfully at start up, it is no guarantee that it will remain registered. Network issues might cause the client to be removed from the network. In this case the client will automatically attempt to reconnect, but our code above will not notice this. In many cases this is not a problem, as the client will recover most situations in the background once a

connection is re-established. If you wish to monitor what is happening and perform some actions in your code, you can register handlers to be notified when the client gets registered to or deregistered from the network.

Copy `basic_client.py` as `monitor.py` and replace the `run_client` function as follows:

monitor.py

```
1 Unresolved directive in ../partials/connect.adoc - include::src/connect_client/monitor.py[tag=show]
```

```
(venv)$ python monitor.py --log-config log-config.yml
INFO    : silkwave.async_client          : Connected to SILKWAVE as net:cshub:0670227473
INFO    : pycs.client.registration       : Registration accepted
INFO    : pycs.client.registration       : Deregistration successful
INFO    : silkwave.async_client          : Disconnected
```

2.5. Explicit start and stop

The `async with` mechanism is a convenient way to start and stop a client, as it safely handles exceptions and makes sure that the client is always shutdown cleanly. However there are times when you want to explicitly start and stop a client. The client provides `start()` and `stop()` functions for this purpose. The equivalent code to the basic `async with` version in [Section 2.1, “Basic connection”](#) would be as show below.

Copy `basic_client.py` as `explicit.py` and replace the `run_client` function as follows:

explicit.py

```
1 Unresolved directive in ../partials/connect.adoc - include::src/connect_client/explicit.py[tag=show]
```

```
(venv)$ python explicit.py --log-config log-config.yml
INFO    : silkwave.async_client          : Connected to SILKWAVE as net:cshub:0205992509
INFO    : pycs.client.registration       : Registration accepted
INFO    : pycs.client.registration       : Deregistration successful
INFO    : silkwave.async_client          : Disconnected
```

2.6. CommonServicesNode

The `pycs.client` package also provides a `CommonServicesNode` class, which provides a number of useful functions to help with maintaining capability and status stores, subscribing to messages from other clients, and for managing the lifecycle of different nodes using the same client connection.

We will explore some of these capabilities later in the tutorial, but for now will look at how to subclass this class, and manage a single node lifecycle.

Copy `basic_client.py` as `cs_node.py`, add the `ExampleNode` class and replace the `run_client` function as follows:

cs_node.py

```
1 Unresolved directive in ../partials/connect.adoc - include::src/connect_client/cs_node.py[tag=show]
```

```
(venv)$ python cs_node.py --log-config log-config.yml
INFO    : silkwave.async_client          : Connected to SILKWAVE as net:cshub:1921012869
INFO    : pycs.client.registration       : Registration accepted
INFO    : pycs.client.registration       : Deregistration successful
INFO    : silkwave.async_client          : Disconnected
```

Here, we can see that the node is started, and then a separate task is spawned to go off and do some work, and the `async wait` block sits waiting for the node to finish with `await node.finished()`. Once that task has done what it needs to do, it requests that the node is stopped by calling `stop()`, which causes the `finished()` to return, and allows the client to shutdown by exiting the `async with` block.

This is a common pattern for nodes on the network. Short-lived nodes, eg taskers might fire off a command, wait for a response and then exit. Long-lived nodes, eg sensors, might start up and continue indefinitely, never calling the `stop()` function.

Chapter 3. Streetlights Example Definitions

Connecting a client to the network is all very well, but not very useful in itself. We need to be able to send and receive messages to other clients on the network, and possibly advertise our capability and current status. We will therefore define a simple 'Streetlight' system allowing a control application to monitor the light levels measured by a number of streetlights, and to request that these streetlights turn themselves on and off as the light levels change. This system will involve two separate applications, the streetlights themselves, and a control system.

The streetlight application will define a number of streets, each of which have a number of streetlights. Each Streetlight has a light sensor to provide light intensity readings, and a lamp which may be on, off or disabled. This application will report the above to the network in Capability and Status messages, and respond to requests to turn the lights on and off.

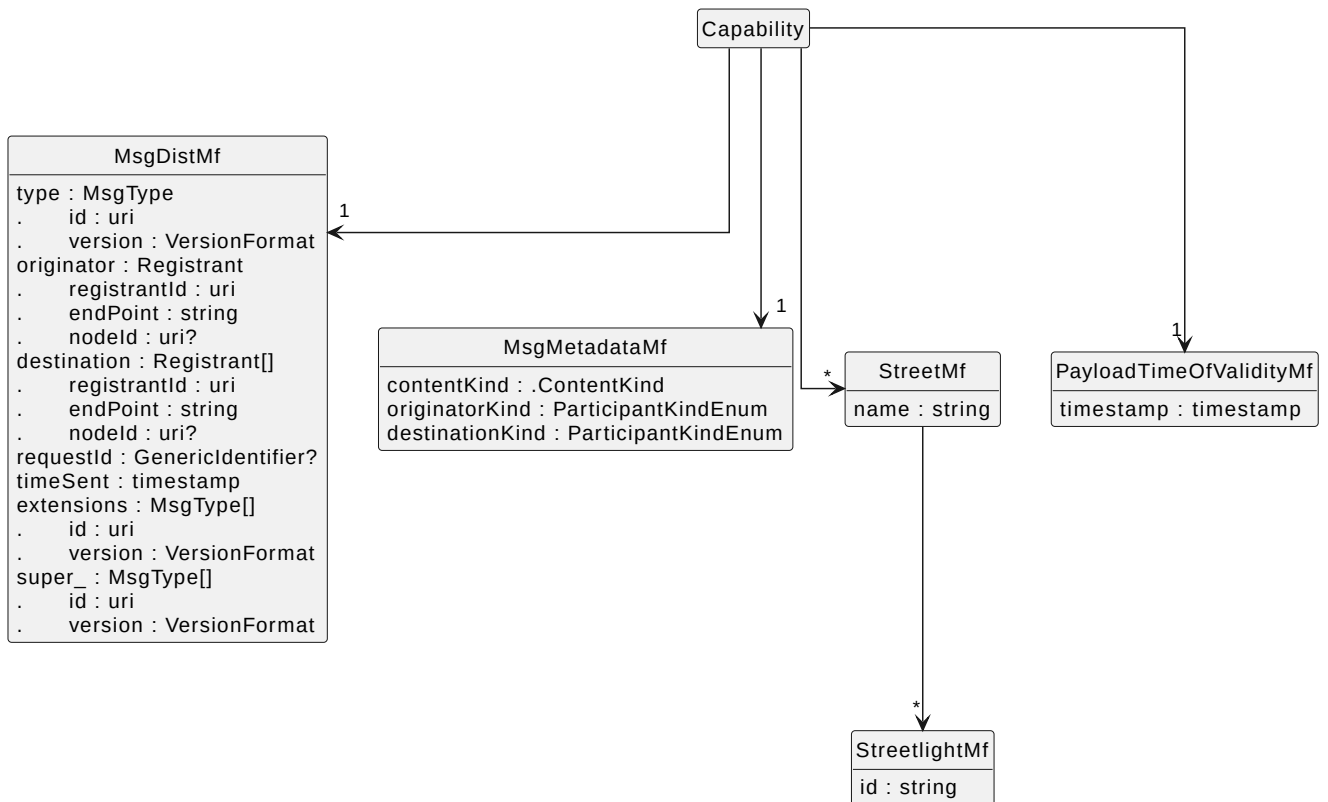
The control application monitors the light levels reported by the streetlights by subscribing to the Capability and Status messages, and issues commands them to turn on or off when the light levels meet predefined thresholds. It can send a command to turn on a whole street, or individual streetlights within that street.

The microformat and message definitions are in the [pycs-tutorial/definitions](#) folder, and define the following messages.

3.1. csmf.msg.tutorial

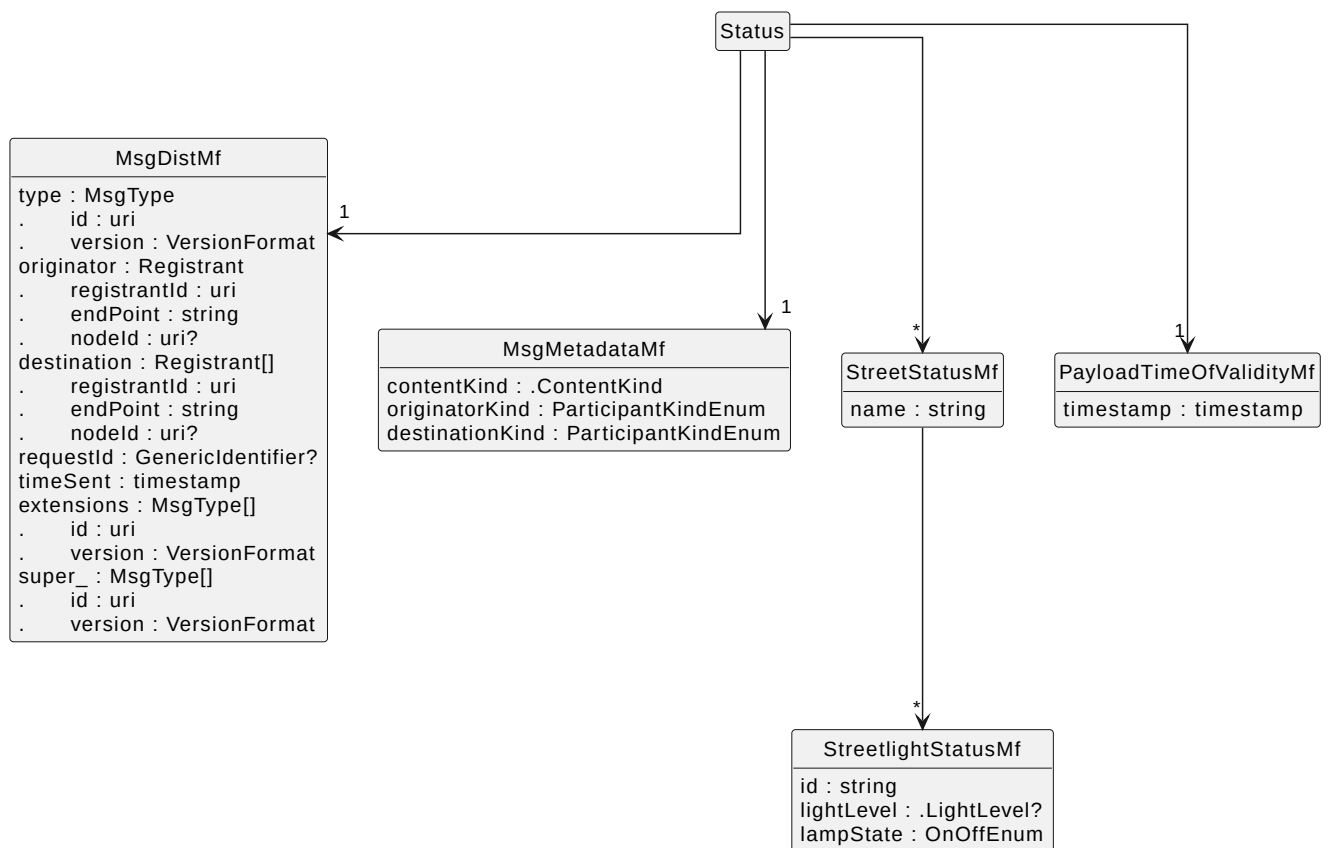
3.1.1. Capability

Message urn:csmf:tutorial:capability version 1.0



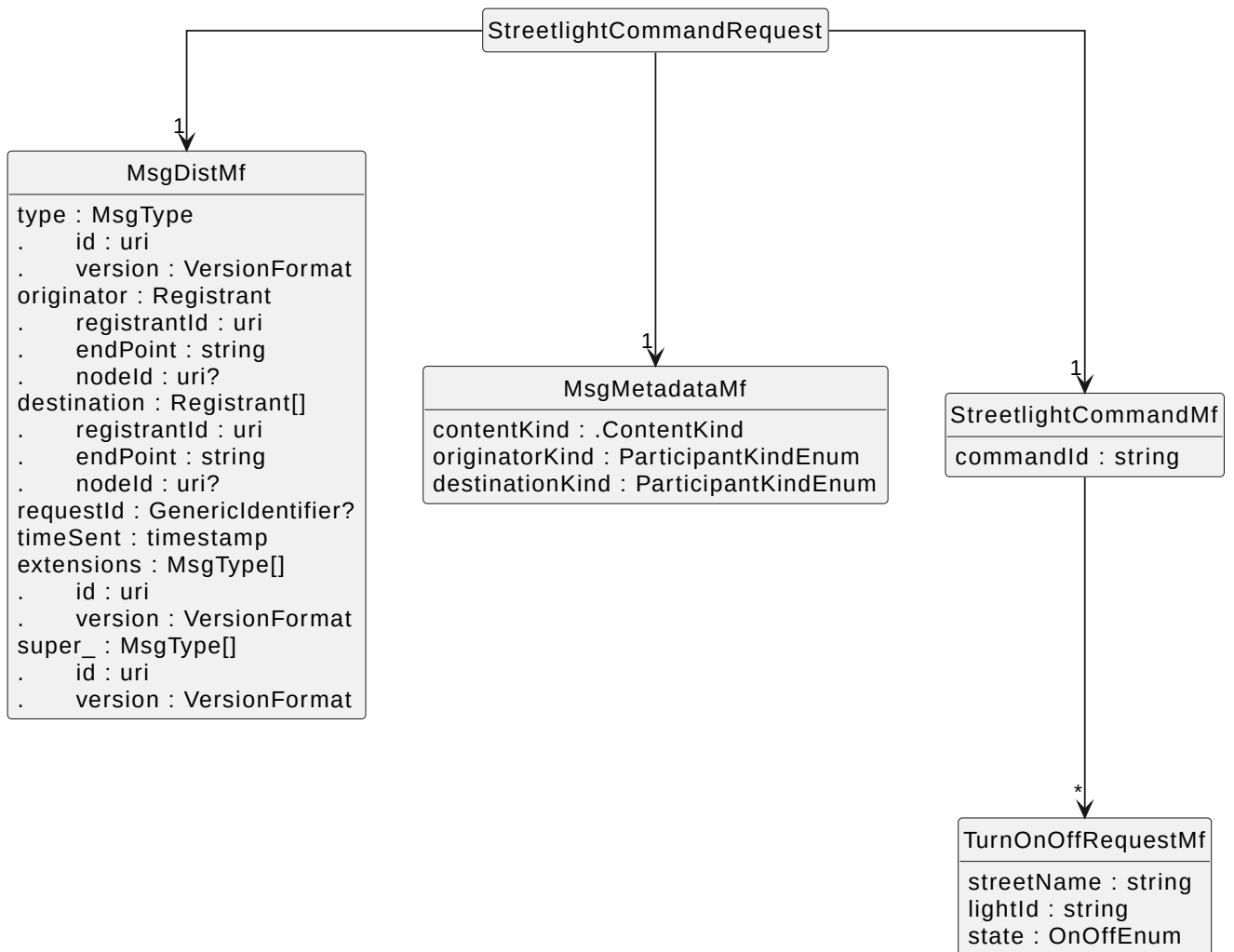
3.1.2. Status

Message urn:csmf:tutorial:status version 1.0



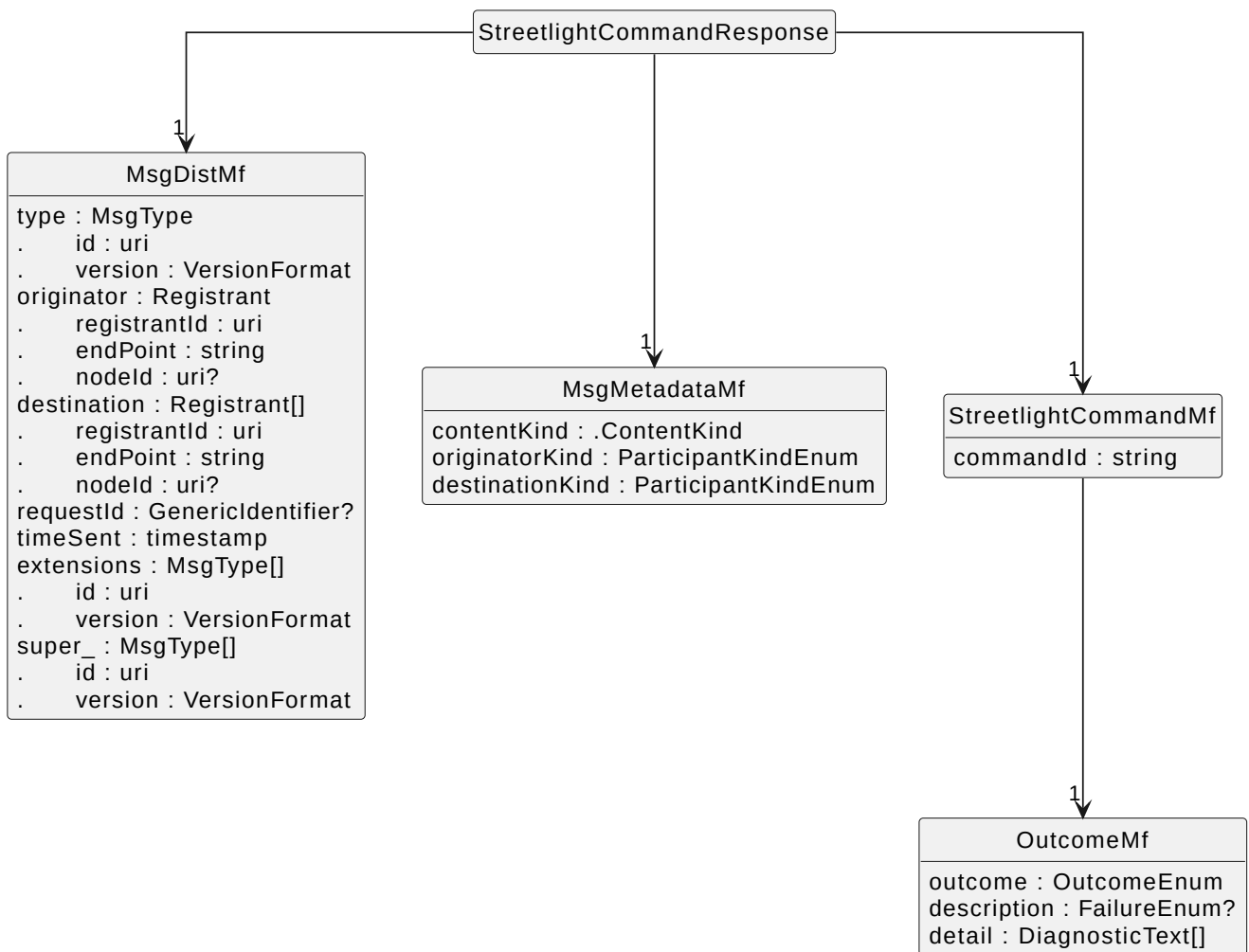
3.1.3. StreetlightCommandRequest

Message urn:csmf:tutorial:streetlightcommandrequest version 1.0



3.1.4. StreetlightCommandResponse

Message urn:csmf:tutorial:streetlightcommandresponse version 1.0



Chapter 4. Generating a message set

4.1. Generating Microformats

First, we generate the microformat code from the microformat definitions. By convention, microformats are generated into sub-modules of `csmf.mf`, so we will generate into `csmf.mf.tutorial`, using `-m` and name the pip package similarly using `-p`. It is possible to generate multiple python modules into one overarching pip package by running multiple generate commands to the same output directory and specifying the same pip package name with the `-p` parameter. Our definitions are dependent on definitions in the `csmf.mf.common` module in the `aoco-533-common-domain` package, so we need to tell the generator to import this module during generation using `-i`, and also to add the package to the package requirements using `-r`. Note that any packages included in this way must be available in the virtual environment at generation time, but in this case `aoco-533-common-domain` is already installed as a dependency of `pys.client`.

```
(venv)$ pys-generate -v 0.0 -o generated/mf -m csmf.mf.tutorial -p csmf.mf.tutorial -i csmf.mf.common -r aoco-533-common-domain pys-tutorial/definitions/microformats/*.mfd
```

We now need to add these generated messages in to our virtual environment to make them available for use, both by subsequent generation steps and by our code. In production, this would normally be done by pushing the package to a pip repository, and installing from there. For the purposes of this tutorial we will add the package to the environment directly from the generated code. We will add it as an editable package so that any re-generations following updates to the definitions are picked up automatically. This generally makes life easier during development.

```
(venv)$ pip install -e generated/mf
```

4.2. Generating Messages

Once we have the microformat code generated, we can generate the message code. In addition to the `csmf.mf.common` package, we now need to tell the code generator to include the `csmf.mf.tutorial` package that we generated in the previous setp. Again, all packages used in this way need to be available in the virtual environment. For message generation the final sub-module name containing each message is derived from the uri specified in the message definition. For this reason, the module name that we pass to the generator is the parent module. By convention, this is `csmf.msg`.

```
(venv)$ pys-generate -v 0.0 -o msggen -m csmf.msg -p csmf.msg.tutorial -i csmf.mf.common -r aoco-533-common-domain -i csmf.mf.tutorial -r csmf.mf.tutorial pys-tutorial/definitions/messages/*.cm
```

Again, we add the generated code to our virtual environment as an editable package.

```
(venv)$ pip install -e generated/msg
```

Chapter 5. Streetlights and Control applications

Now that we have our messages generated we can start to put together the Streetlights application, which represents the configuration of streetlights on the network, and the Control application, which monitors the light levels and turns lamps on and off as necessary.

The directory `pys-tutorial/src/pys/tutorial` contains all finished code needed to run the two applications, but we will build them up gradually in this tutorial, adding code as we add additional functionality.

Create a directory `pys/tutorial`, and copy the `cs_node.py` from earlier in twice as `streetlights.py` and `control.py`. These will form the basis for our two applications. Also create an empty `__init__.py` in the directory to turn this directory into a python module.

```
(venv)$ mkdir pys/tutorial
(venv)$ cp cs_node.py pys/tutorial/streetlights.py
(venv)$ cp cs_node.py pys/tutorial/control.py
(venv)$ touch pys/tutorial/__init__.py
```

5.1. Streetlights Application

Our streetlight application will eventually be configured by loading in a configuration from the config file specified on the command line. We can specify a schema for this in the `@client_application` decorator. We also change the `cs_id` and `name` parameters to something more appropriate.

Our client will need a copy of the config object, so we take this from the arguments supplied by `@client_application` and pass it through to `run_client`.

pys/tutorial/streetlights.py

```
1 Unresolved directive in ../partials/applications.adoc - include::src/pys/tutorial/streetlights.py[tags=main]
```

The `config_schema_pkg` and `config_schema` parameters are telling `@client_application` to look for the schema in the current package, so we need to supply this file. Copy the schema `streetlights-schema.yml` into `pys/tutorial`.

Replace the `ExampleNode` class with the following `Streetlights` class, and instantiate this from `run_client`, passing the new config though. The config object attributes will match properties in our schema, and all the data we need is contained in the `config.streetlights` attribute.

pys/tutorial/streetlights.py

```
1 Unresolved directive in ../partials/applications.adoc - include::src/pys/tutorial/streetlights.py[tags
=application;run_client]
```

5.2. Control Application

Because it is likely that multiple control applications might be running at the same time we will append a UUID to the network id to ensure that they are all unique. We should also report that we are a `tasker` in the `MsgMetadataMf`, so pass `kind` appropriately.

Our control application will also be configured from a config file, with its own schema. Copy the schema `streetlights-schema.yml` into `pys/tutorial` to enable this. However, this time rather than taking the config object directly, we will add command line parameters for the configurable data, and use the facilities available in `@application_client` to use the config file to set the values when the command line parameter is not supplied. To this end, we add two `@click.option` decorators to `main()`, and use `config_defaults` to set the default values for these if provided in the config file. Note that these decorators must appear before the `@client_application` decorator.

pys/tutorial/control.py

```
1 Unresolved directive in ../partials/applications.adoc - include::src/pys/tutorial/control.py[tags=main]
```

Similarly to the Streetlights application, replace `ExampleNode` with `StreetlightsControl`, and call this from `run_client`, but this time passing in the threshold parameters directly rather than via the config.

pys/tutorial/control.py

```
1 Unresolved directive in ../partials/applications.adoc - include::src/pys/tutorial/control.py[tags
=application;run_client]
```

Chapter 6. Capability and Status

6.1. Publishing

The Streetlights application needs to publish its capability and status to the CS network.

We add capability and status members into our `Streetlight` class, using the `CommonServicesNode.create_capability` and `CommonServicesNode.create_status` functions to supply Capability and Status messages with the `MsgDistMf` and `MsgMetadataMf` microformats correctly initialised. Once the application has started up, the `started()` function will be called, where we can notify the Capability and Status services of their data. The `CommonServicesNode.update_capability` and `CommonServicesNode.update_status` functions populate the local capability and status stores with our current state, and then send out and update to the CS network.

pycs/tutorial/streetlights/streetlights.py

```
1 Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/streetlights.py[tags=cap_sta_publish]
```

Run the application from the command line.

```
(venv)$ python -m pycs.tutorial.streetlights --log-config log-config.yml
INFO    : silkwave.async_client          : Connected to SILKWAVE as net:cshub:1575410301
INFO    : pycs.client.registration       : Registration accepted
INFO    : pycs.client.LocalMicroformatStore : Created full update
INFO    : pycs.client.LocalMicroformatStore : Created full update
```

Once the application has started, check that the Status and Capability for our client have been received by the CS Network by examining the CS Monitor.

The Capability and Status services should have received our updates, so the node counts on the CS Monitor for Local Cap & Sta should have increased from zero to one. If you are connecting to a hub that has other clients, you may see different numbers here, but they should increment by one as you connect.

[CS Monitor Node Counts] | *screenshots/cs-monitor-node-counts.png*

Running the CS-Capabilities service, 'list local cap' command on the Admin Interface should show the capability message that we published to the network. Again, if your hub has more clients you may see extra MfDoc elements for the other clients. In this case you could use the 'get cap' command, using `urn:csmf:pycs:tutorial:streetlights` as the regId in the arguments field.

Capability

```
Unresolved directive in ../partials/capsta.adoc - include::messages/publish_capability.xml[]
```

Running the similar command on the CS-Status service should show the Status message.

```
Unresolved directive in ../partials/capsta.adoc - include::messages/publish_status.xml[]
```

We can now shutdown the application cleanly with a keyboard interrupt.

```
^C
INFO    : pycs.client.capability          : Purge Successful
INFO    : pycs.client.status              : Purge Successful
INFO    : pycs.client.registration        : Deregistration successful
INFO    : silkwave.async_client             : Disconnected
```

The client and its Capability and Status should disappear from the CS-Monitor.

6.2. Populating

While we have now advertised our capability and status to the network, we have not populated any useful information in there. There are many ways we could do this in real life, maybe querying some hardware, or a configuration UI, but for our purposes here we will use the configuration file. The `run_client` function is passing us the `streetlights` object from the main config, so we use this to populate our Capability and Status. We will also initialise a `status_lookup` attribute to easily find the StreetlightStatusMf microformats, which will come in handy in later on. Add add a new function `configure` to populate `self.streets`, and invoke it with the supplied config in the class `__init__` function .

pycs/tutorial/streetlights.py

```
1 Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/streetlights.py[tags
=cap_sta_populate_imports]
```

pycs/tutorial/streetlights.py, class `Streetlights`

```
1 Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/streetlights.py[tags=cap_sta_populate]
```

pycs/tutorial/streetlights.py, function `Streetlights.__init__`

```
Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/streetlights.py[tags=capsta_populate_init]
```

Let's check everything is working by providing a minimal config file to set up a single Street with a single Streetlight.

minimal_config.yml

```
1 Unresolved directive in ../partials/capsta.adoc - include::src/minimal_config.yml[]
```

```
(venv)$ python -m pycs.tutorial.streetlights --log-config log-config.yml --config minimal_config.yml
```

```

INFO : silkwave.async_client : Connected to SILKWAVE as net:cshub:1635775766
INFO : pycs.client.registration : Registration accepted
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.client.LocalMicroformatStore : Created full update
^C
INFO : pycs.client.capability : Purge Successful
INFO : pycs.client.status : Purge Successful
INFO : pycs.client.registration : Deregistration successful
INFO : silkwave.async_client : Disconnected

```

This time when we examine the Capability and Status in the CS Monitor, we see extra microformats for the Street and Streetlight we added.

Capability

```

Unresolved directive in ../partials/capsta.adoc - include::messages/populate_capability_mfs.xml[]

```

Status

```

Unresolved directive in ../partials/capsta.adoc - include::messages/populate_status_mfs.xml[]

```

6.3. Updating

In a real world situation, the sensors on the streetlamps would report the ambient light levels, but we need to simulate that for this tutorial. We will introduce a new class `LevelAdjuster`, which can be configured to adjust light levels between a min and max at a fixed interval. The details of this are unimportant, other than the fact that it will periodically change the light level on each streetlamp, and after each round of changes, report that update to the CS Network.

pycs/tutorial/streetlights.py

```

1 Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/streetlights.py[tag=level_adjuster]

```

In the `Streetlights.__init__` constructor, add a member to hold the adjuster

pycs/tutorial/streetlights.py, function `Streetlights.__init__`

```

1 Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/streetlights.py[tag=adjuster_init]

```

and in `Streetlights.configure` create an instance from the config if supplied.

pycs/tutorial/streetlights.py, function `Streetlights.configure`

```

1 Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/streetlights.py[tag=adjuster_populate]

```

Once the process is started, we should start the adjuster in `Streetlights.start`,

pycs/tutorial/streetlights.py, function `Streetlights.start`

```
1 Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/streetlights.py[tag=adjuster_start]
```

and make sure it is shut down cleanly in `Streetlights.closing`

pycs/tutorial/streetlights.py, class `Streetlights`

```
1 Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/streetlights.py[tag=adjuster_stop]
```

Create a new config file to configure the adjuster.

adjust_config.yml

```
1 Unresolved directive in ../partials/capsta.adoc - include::src/adjust_config.yml[]
```

and run the application using this configuration. As you can see from the log output, the level adjuster is periodically changing the light level reported by the streetlight. This is causing the local status store to send out updates to the network. Because only a small section of the full Status message is updated, an abbreviated update can be sent. If the update had caused a significant proportion of the message to change, then a full update would have been sent.

You should see the values for the light level change in the CS Montitor status display.

```
(venv)$ python -m pycs.tutorial.streetlights --log-config log-config.yml --config adjust_config.yml
INFO : silkwave.async_client : Connected to SILKWAVE as net:cshub:0582459926
INFO : pycs.client.registration : Registration accepted
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.application: Starting adjuster
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 0.0415452 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 0.338826 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 1.15852 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 34.2623 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 2023.4 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 11291.8 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 2307.2 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 20.9372 lux
```

```

INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 5.78167 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 0.0991711 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
^C
INFO : pycs.tutorial.streetlights.application: Stopping adjuster
INFO : pycs.client.status : Purge Successful
INFO : pycs.client.capability : Purge Successful
INFO : pycs.client.registration : Deregistration successful
INFO : silkwave.async_client : Disconnected

```

6.4. Subscribing

Our Control application needs to monitor the published light levels. To do this, it must first subscribe to the Capability and Status messages issued by the network.

pycs/tutorial/control.py

```

1 Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/control.py[tag=subscribe_imports]

```

pycs/tutorial/control.py, class `StreetlightControl`

```

1 Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/control.py[tag=subscribe]

```

Our application is now subscribed to Capability and Status messages, and the `client.capability_service` and `client.status_service` remote stores will be receiving updates, and could be polled and queried for the latest data. However, it would be much better to be notified of any changes, and the `pycs.client` package provides decorators to easily enable this.

The `@notify_status` decorator causes the function to be used as the callback for any Status change notifications. It is equivalent to calling `self.add_status_notification(self.receive_sensor_status, Status)` in the `started()` function.

We are not interested in the capability message at this stage, but if we were we would decorate the service with `@notify_capability(Capability)`.

When we receive a Status message, we will output the status of each streetlight to the log.

pycs/tutorial/control.py, class `StreetlightControl`

```

1 Unresolved directive in ../partials/capsta.adoc - include::src/pycs/tutorial/control.py[tag=notify_status]

```

Run the Streetlights application in the background (or in a separate terminal), and then run up the control application.

You should see the control application subscribe to both Capability and Status messages, request an

initial full update from the hub, and then continue to receive a number of abbreviated updates. As it receives each Status update it logs the current status of the system. Note however, that it does not necessarily receive every status message issued. This is because the Common Service Status Service may rate limit the messages that it forwards on.

```
(venv)$ python -m pycs.tutorial.streetlights --config adjust_config.yml &> streetlights.log &

(venv)$ python -m pycs.tutorial.control --log-config log-config.yml
INFO : silkwave.async_client : Connected to SILKWAVE as net:cshub:0757619514
INFO : pycs.client.registration : Registration accepted
INFO : pycs.client.capability : Subscribing
INFO : pycs.client.status : Subscribing
INFO : pycs.client.status : Subscription Successful
INFO : pycs.client.status : Requesting full update
INFO : pycs.client.status : Received Subscription Update
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:44.719543+00:00
INFO : pycs.tutorial.control :
-----
INFO : pycs.tutorial.control : Festive Road:
INFO : pycs.tutorial.control : 52: off 0.0213387 lux
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:44.719543+00:00
INFO : pycs.tutorial.control :
-----
INFO : pycs.tutorial.control : Festive Road:
INFO : pycs.tutorial.control : 52: off 0.0213387 lux
INFO : pycs.client.capability : Subscription Successful
INFO : pycs.client.capability : Requesting full update
INFO : pycs.client.capability : Received Subscription Update
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:44.678423+00:00
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:44.678423+00:00
INFO : pycs.client.capability : Received Subscription Update
INFO : pycs.client.status : Received Subscription Update
INFO : pycs.client.status : Received Subscription Update
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:47.815703+00:00
INFO : pycs.tutorial.control :
-----
INFO : pycs.tutorial.control : Festive Road:
INFO : pycs.tutorial.control : 52: off 300.761 lux
INFO : pycs.client.status : Received Subscription Update
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:50.929586+00:00
INFO : pycs.tutorial.control :
-----
INFO : pycs.tutorial.control : Festive Road:
INFO : pycs.tutorial.control : 52: off 1285.19 lux
^C
INFO : pycs.client.registration : Deregistration successful
INFO : silkwave.async_client : Disconnected

(venv)$ jobs
[1] + running python -m pycs.tutorial.streetlights --config adjust_config.yml &>

(venv)$ kill %1
[1] + 46840 done python -m pycs.tutorial.streetlights --config adjust_config.yml &>
```

Chapter 7. Requests and Responses

7.1. Sending a Request

Our Control application is currently not doing very much controlling, only monitoring the current status. We will now add code to send out requests to the Streetlights application to turn lights on and off when the light levels reach configured thresholds.

We already have code in the application to set values for the thresholds from command line parameters or from the config file, so now we just need to monitor the light levels and send out requests as appropriate.

We would like to send out the request as soon as a status message comes in reporting a light level beyond our thresholds, so we can add another function decorated with `@notify_status` to do this.

pycs/tutorial/control.py

```
1 Unresolved directive in ../partials/request.adoc - include::src/pycs/tutorial/control.py[tag=request_changes_imports]
```

pycs/tutorial/control.py, class `StreetlightControl`

```
1 Unresolved directive in ../partials/request.adoc - include::src/pycs/tutorial/control.py[tag=request_changes]
```

First we check to see whether any thresholds have been set, as there is no point continuing if not. Then we start to build up a `StreetlightCommandRequest` message to send out, adding a `TurnOnOffRequestMf` for any light that needs a change. Finally, if there were any changes needed we send out the request and wait for a response to come back.

7.2. Handling Requests

Our Streetlights application now needs to handle these requests and send a response back.

The easiest way to subscribe to a message if the subscription should persist through the life of the client is to use the `@subscriber` decorator on a function. Whenever any message of the correct type is received and addressed to the current node, the function will be called. Using this decorator is equivalent to calling `self.subscribe_callback` in the `started()` function. Please see the main `pycs.client` documentation for other methods of subscribing to messages.

When we receive the request, we will loop around each of the `TurnOnOffRequestMf` microformats, find the correct `StreetlightStatusMf` using the lookup we handily populated earlier, and update the lamp state accordingly, and send the `StreetlightCommandResponse` back to the sender. If any changes were made, then we also send out a Status update to the network.

pycs/tutorial/streetlights.py

```
1 Unresolved directive in ../partials/request.adoc - include::src/pycs/tutorial/streetlights.py[tag=subscribe_request_imports]
```

```
1 Unresolved directive in ../partials/request.adoc - include::src/pycs/tutorial/streetlights.py[tag=subscribe_request]
```

Run the Control and Streetlights applications in separate terminals, and watch the status change from **off** to **on** and back again as the light levels change.

```
(venv)$ python -m pycs.tutorial.control --log-config log-config.yml --turn-on-threshold 80 --turn-off-threshold 160
INFO : silkwave.async_client : Connected to SILKWAVE as net:cshub:1681233045
INFO : pycs.client.registration : Registration accepted
INFO : pycs.client.capability : Subscribing
INFO : pycs.client.status : Subscribing
INFO : pycs.client.status : Subscription Successful
INFO : pycs.client.status : Requesting full update
INFO : pycs.client.status : Received Subscription Update
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:56.757366+00:00
INFO : pycs.tutorial.control :
-----
INFO : pycs.tutorial.control : Festive Road:
INFO : pycs.tutorial.control : 52: off 0 lux
INFO : pycs.tutorial.control : Requesting Festive Road 52 to turn on
INFO : pycs.tutorial.control : Requesting changes
INFO : pycs.client.capability : Subscription Successful
INFO : pycs.client.capability : Requesting full update
INFO : pycs.client.capability : Received Subscription Update
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:56.737159+00:00
INFO : pycs.tutorial.control : Request accepted
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:56.737159+00:00
INFO : pycs.client.status : Received Subscription Update
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:57.811697+00:00
INFO : pycs.tutorial.control :
-----
INFO : pycs.tutorial.control : Festive Road:
INFO : pycs.tutorial.control : 52: on 0.209012 lux
INFO : pycs.client.capability : Received Subscription Update
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:56.737159+00:00
INFO : pycs.client.status : Received Subscription Update
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:26:59.871208+00:00
INFO : pycs.tutorial.control :
-----
INFO : pycs.tutorial.control : Festive Road:
INFO : pycs.tutorial.control : 52: on 20.1934 lux
INFO : pycs.client.status : Received Subscription Update
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:27:01.932035+00:00
INFO : pycs.tutorial.control :
-----
INFO : pycs.tutorial.control : Festive Road:
INFO : pycs.tutorial.control : 52: on 28862.2 lux
INFO : pycs.tutorial.control : Requesting Festive Road 52 to turn off
INFO : pycs.tutorial.control : Requesting changes
INFO : pycs.tutorial.control : Request accepted
INFO : pycs.client.status : Received Subscription Update
INFO : pycs.client.RemoteMicroformatStore : Received full update at 2024-08-08 12:27:05.023422+00:00
INFO : pycs.tutorial.control :
-----
INFO : pycs.tutorial.control : Festive Road:
INFO : pycs.tutorial.control : 52: off 1.33823 lux
INFO : pycs.tutorial.control : Requesting Festive Road 52 to turn on
INFO : pycs.tutorial.control : Requesting changes
INFO : pycs.tutorial.control : Request accepted
^C
INFO : pycs.client.registration : Deregistration successful
```

```
INFO : silkwave.async_client : Disconnected
```

```
(venv)$ python -m pycs.tutorial.streetlights --log-config log-config.yml --config adjust_config.yml
INFO : silkwave.async_client : Connected to SILKWAVE as net:cshub:0707803679
INFO : pycs.client.registration : Registration accepted
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.application: Starting adjuster
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 0.0115863 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.application: Received StreetlightCommandRequest
INFO : pycs.tutorial.streetlights.application: Streetlight Festive Road 52 turned on
INFO : pycs.tutorial.streetlights.application: Sending StreetlightCommandResponse
INFO : pycs.tutorial.streetlights.application: Sending status update
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 0.209012 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 1.3446 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 20.1934 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 573.324 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 28862.2 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 968.344 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.application: Received StreetlightCommandRequest
INFO : pycs.tutorial.streetlights.application: Streetlight Festive Road 52 turned off
INFO : pycs.tutorial.streetlights.application: Sending StreetlightCommandResponse
INFO : pycs.tutorial.streetlights.application: Sending status update
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 35.8324 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 1.33823 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.application: Received StreetlightCommandRequest
INFO : pycs.tutorial.streetlights.application: Streetlight Festive Road 52 turned on
INFO : pycs.tutorial.streetlights.application: Sending StreetlightCommandResponse
INFO : pycs.tutorial.streetlights.application: Sending status update
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
INFO : pycs.client.LocalMicroformatStore : Created full update
INFO : pycs.tutorial.streetlights.level_adjuster: Setting light Festive Road - 52: 0.120075 lux
INFO : pycs.client.LocalMicroformatStore : Created abbreviated update
```

```
INFO : pyes.client.LocalMicroformatStore : Created full update
^C
INFO : pyes.tutorial.streetlights.application: Stopping adjuster
INFO : pyes.client.status : Purge Successful
INFO : pyes.client.capability : Purge Successful
INFO : pyes.client.registration : Deregistration successful
INFO : silkwave.async_client : Disconnected
```

Appendix A: Full Source Code

A.1. Connection

await_reg.py

```
1 Unresolved directive in ../partials/appendix.adoc - include::src/connect_client/await_reg.py[tags=**]
```

basic_client.py

```
1 Unresolved directive in ../partials/appendix.adoc - include::src/connect_client/basic_client.py[tags=**]
```

cs_node.py

```
1 Unresolved directive in ../partials/appendix.adoc - include::src/connect_client/cs_node.py[tags=**]
```

explicit.py

```
1 Unresolved directive in ../partials/appendix.adoc - include::src/connect_client/explicit.py[tags=**]
```

monitor.py

```
1 Unresolved directive in ../partials/appendix.adoc - include::src/connect_client/monitor.py[tags=**]
```

timeout.py

```
1 Unresolved directive in ../partials/appendix.adoc - include::src/connect_client/timeout.py[tags=**]
```

A.1.1. Streetlights

pycs/tutorial/streetlights.py

```
1 Unresolved directive in ../partials/appendix.adoc - include::src/pycs/tutorial/streetlights.py[tags=**]
```

pycs/tutorial/streetlights-schema.yml

```
1 Unresolved directive in ../partials/appendix.adoc - include::src/pycs/tutorial/streetlights-schema.yml[tags=**]
```

minimal_config.yml

```
1 Unresolved directive in ../partials/appendix.adoc - include::src/minimal_config.yml[tags=**]
```

adjust_config.yml

1 Unresolved directive **in** ../partials/appendix.adoc - include::src/adjust_config.yml[tags=**]

full_config.yml

1 Unresolved directive **in** ../partials/appendix.adoc - include::src/full_config.yml[tags=**]

A.1.2. Streetlights Control

pycs/tutorial/control.py

1 Unresolved directive **in** ../partials/appendix.adoc - include::src/pycs/tutorial/control.py[tags=**]

pycs/tutorial/control-schema.yml

1 Unresolved directive **in** ../partials/appendix.adoc - include::src/pycs/tutorial/control-schema.yml[tags=**]